

Hybrid Computing Technology

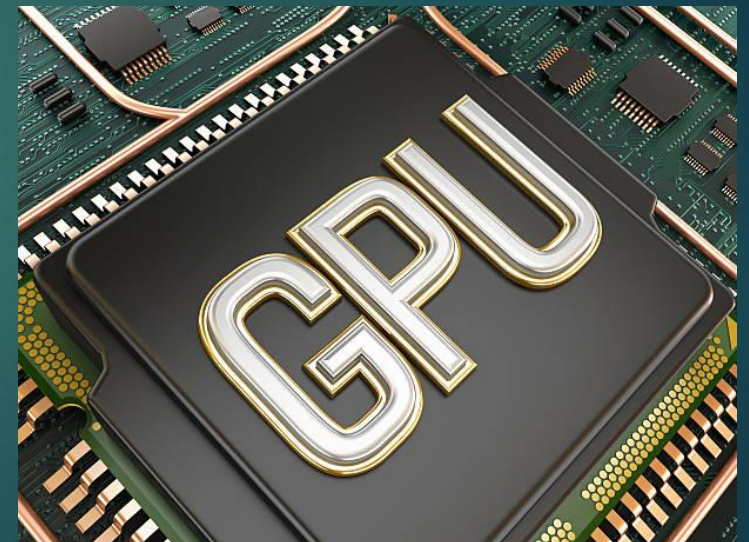
1

GPU-Based Computing Technology

ผศ. อัครินทร์ คุณกิตติ

ASST. PROF. AKHARIN KHUNKITTI

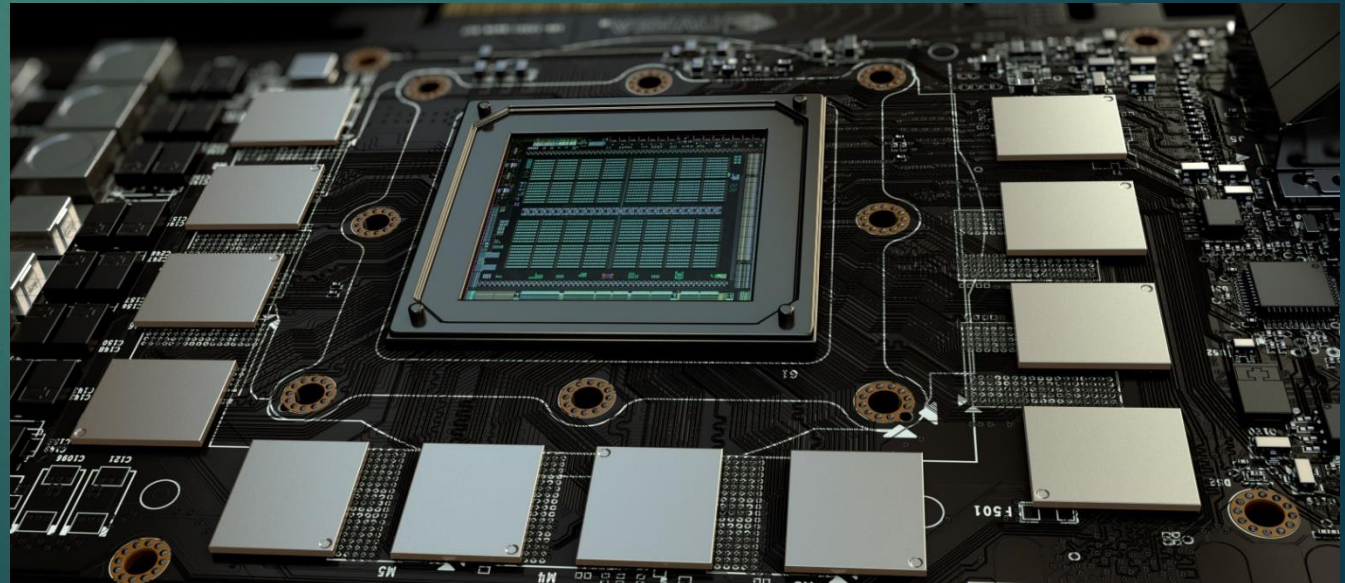
FACULTY OF INFORMATION TECHNOLOGY
KMITL



Topics

2

- ▶ Introduction to Graphics Processing Unit (GPU)
- ▶ GPU Architecture
- ▶ GPU Execution and Programming Model
- ▶ GPU Code and Software Development
- ▶ Conclusion

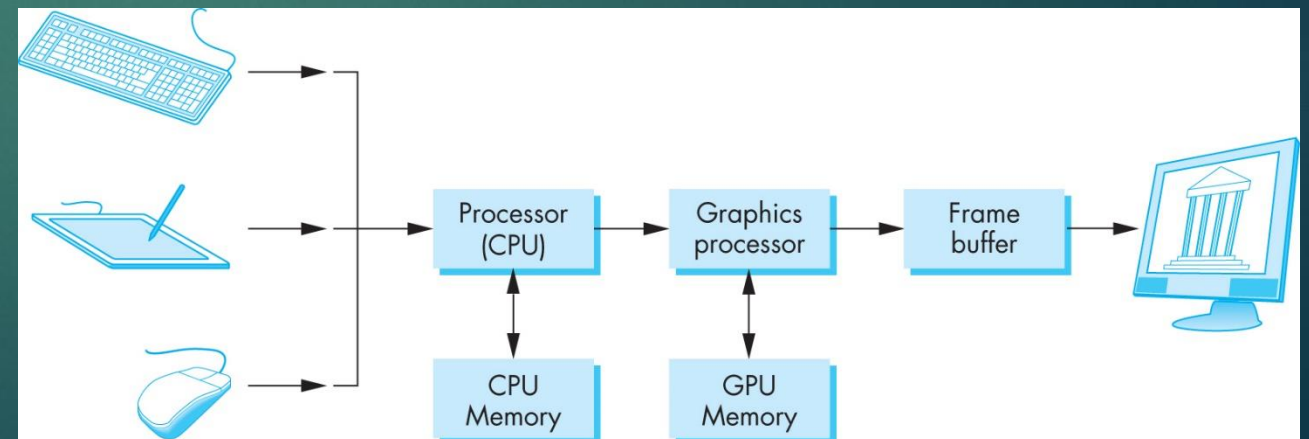
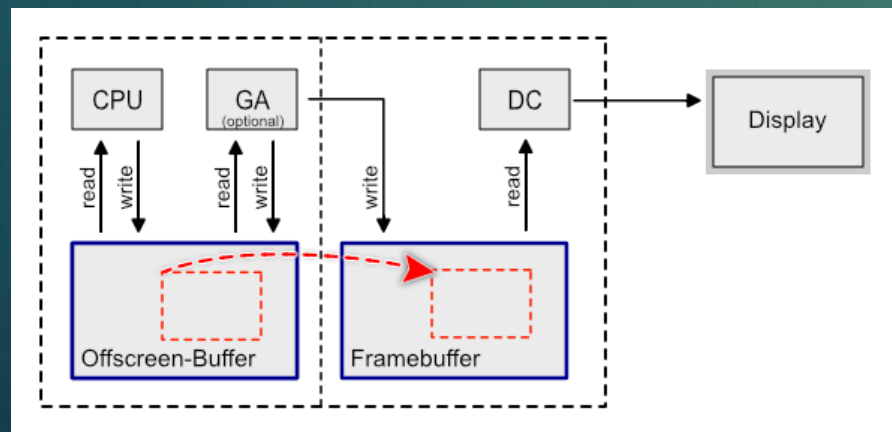
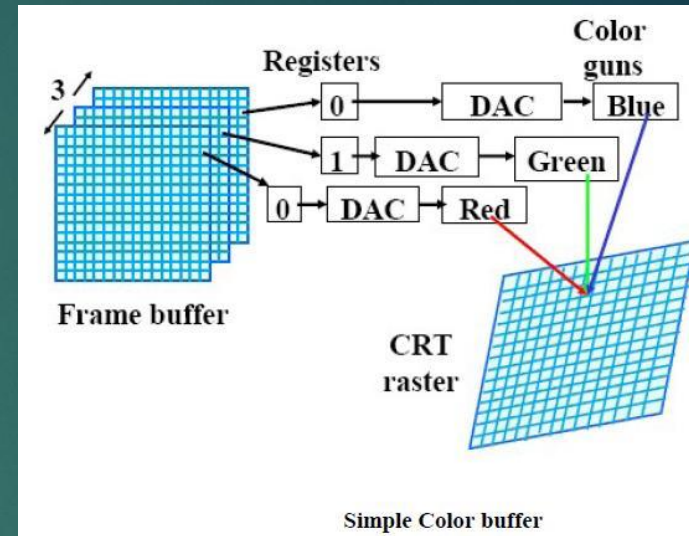


Introduction to GPU

- ▶ GPU – Graphics Processing Unit
- ▶ Start from Display Unit / Card
 - ▶ Video Graphics Array (VGA)
- ▶ Frame Buffer => Display Image Memory
- ▶ Simple Graphic Processing
- ▶ More Graphics Processing



3



Graphics Processing

- ▶ Objects => Display Processor => Frame Buffer
- ▶ More Processing => Graphics Processor
- ▶ Parallel Processors
- ▶ Massively Parallel Processors (MPP)
- ▶ SIMD

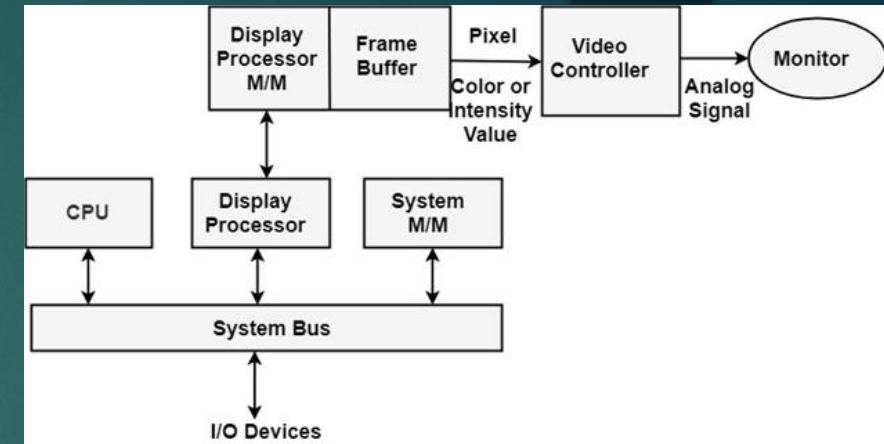
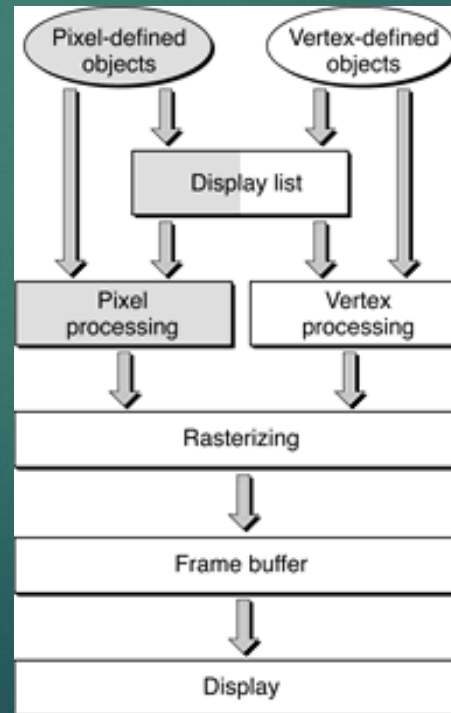
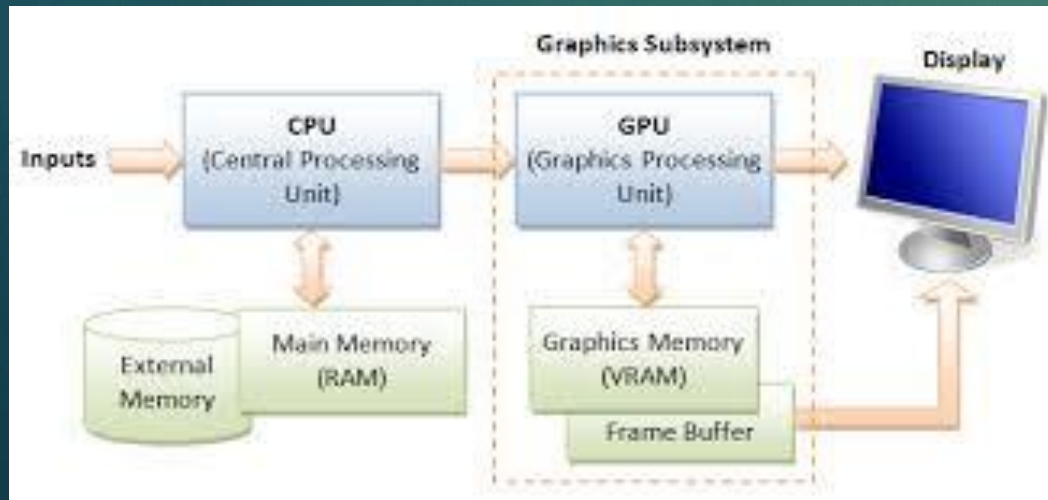
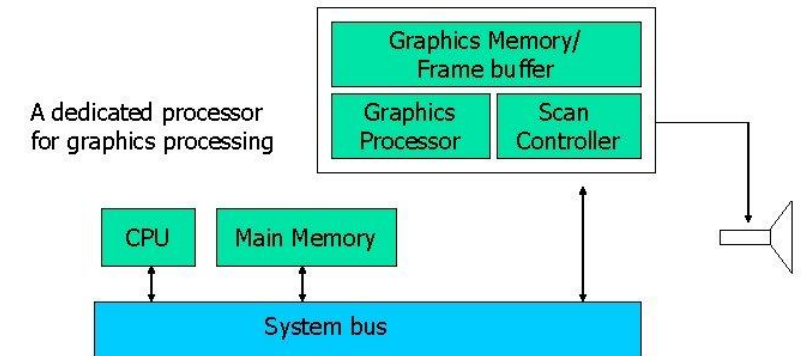


Fig: Architecture of a Raster Display System with a Display Processor

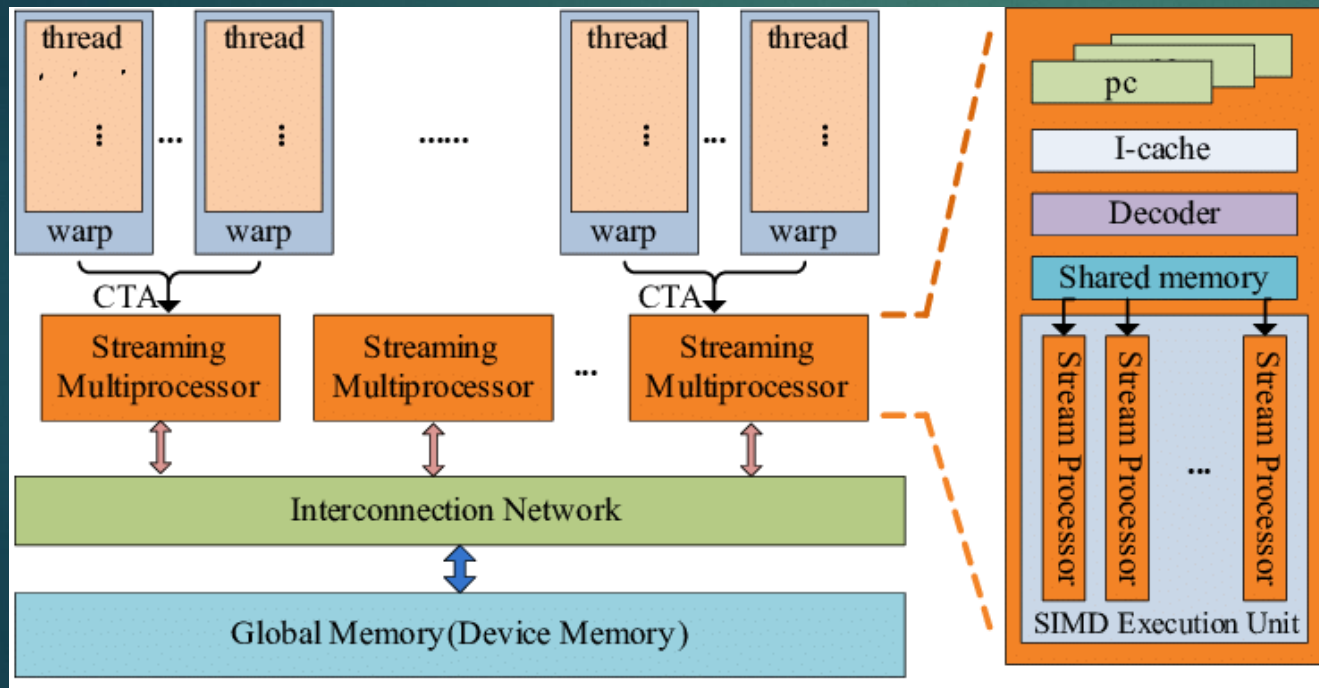
Graphics Accelerator



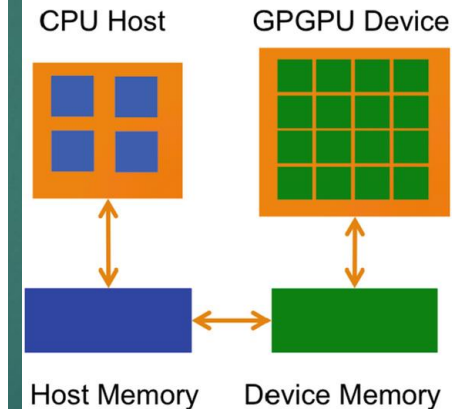
General Purposed GPU (GPGPU)

5

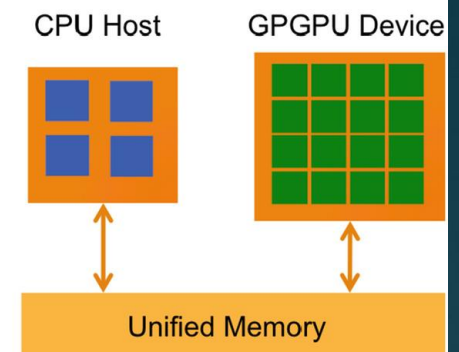
- ▶ Processing Power of GPU => General Number Processing



Traditional View of GPGPU Programming



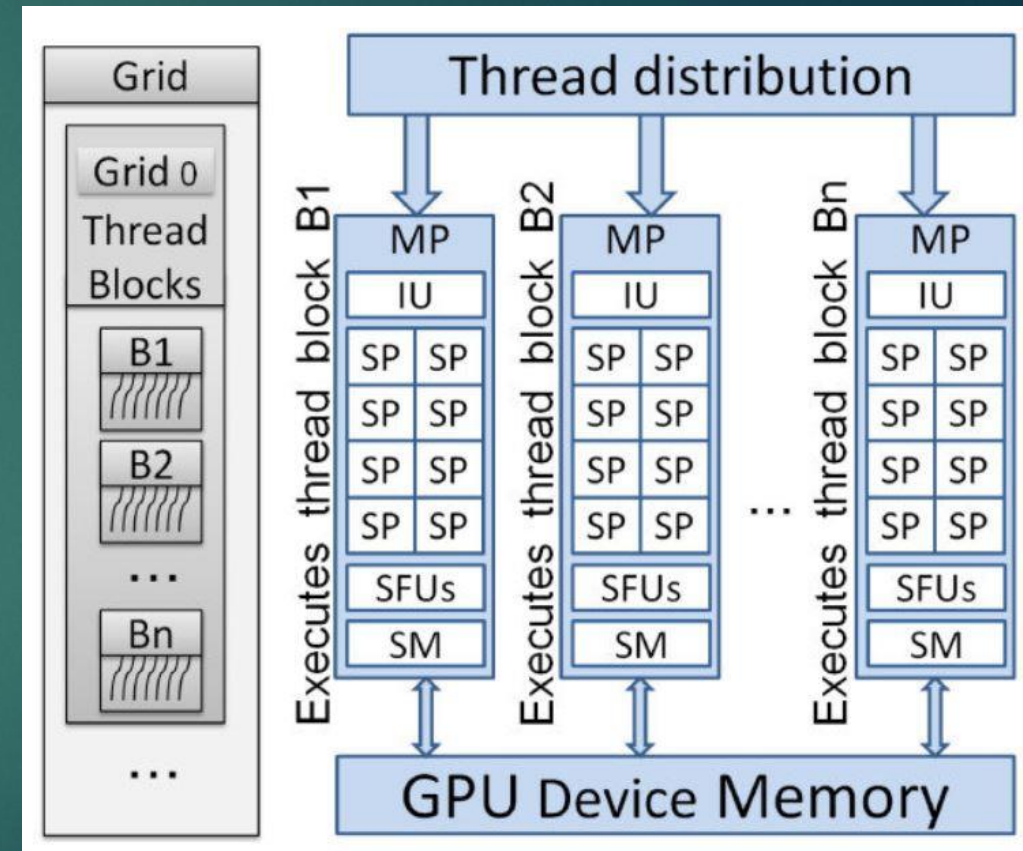
Unified View of GPGPU Programming



GPU Architecture Introduction

6

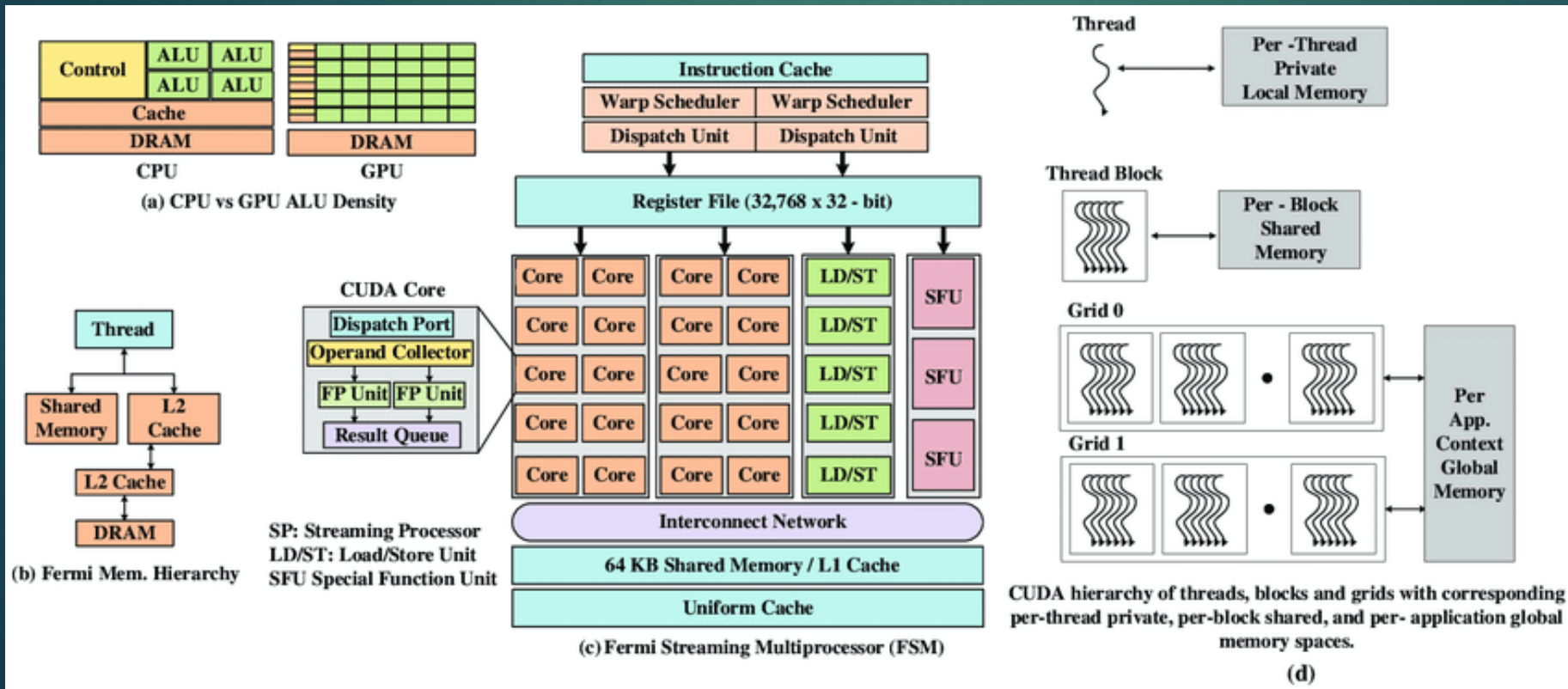
- ▶ A GPU is essentially a commodity stream processor
 - ▶ **Highly parallel** (100s of processor cores)
 - ▶ **Very fast** (>900 GFLOPS of peak performance)
 - ▶ **Operates in a SIMD manner. This is a key restriction**
 - ▶ **Multiple processors operate in lock-step (same instruction) but on different data**
- ▶ GPUs, owing to their **massively parallel architecture**, have been used to accelerate
 - ▶ Image/stream processing, data compression, numerical algorithms
 - ▶ Recently they have been used to accelerate CAD algorithms as well.



GPU Architecture

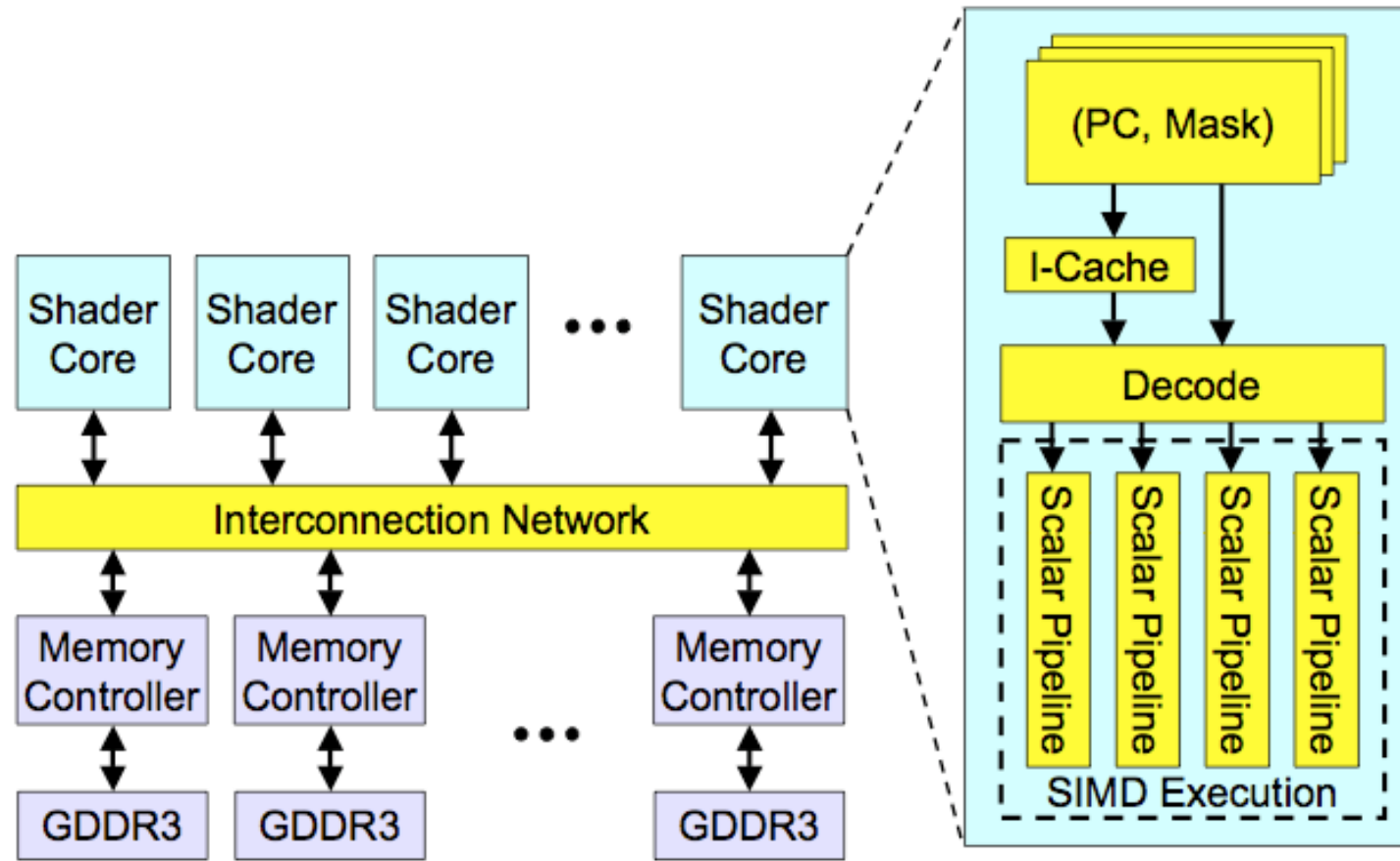
7

- ▶ Physical View
- ▶ Logical / Programming View



High Level Architecture of GPU

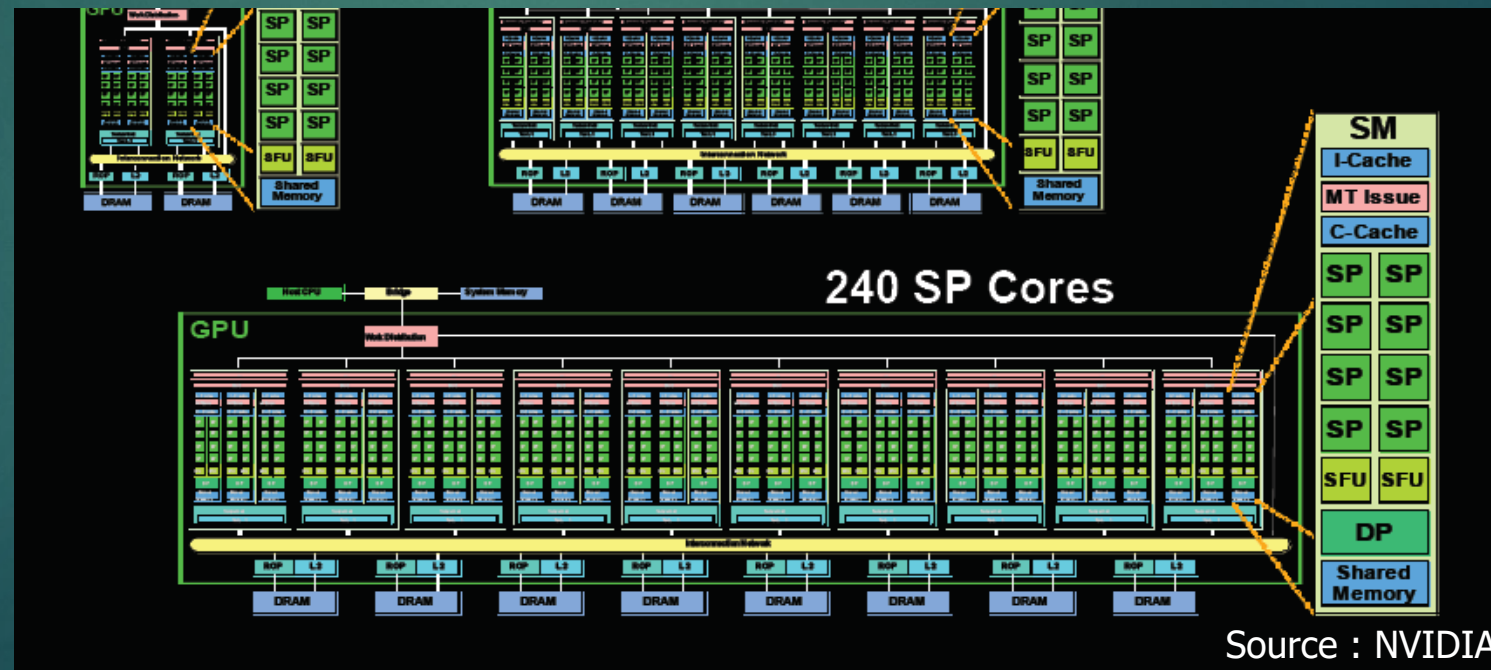
8



GPU Architecture Example

9

- ▶ In the GTX 280, there are 10 Thread Processing Clusters (TPCs)
 - ▶ Each has 3 Streaming Multiprocessors (SMs), which we will refer to as **multiprocessors** (MPs)
 - ▶ Each MP has 8 Streaming Processors (SPs) or Thread Processors (TPs). We will refer to these as **processors**.
 - ▶ **240 processors and 30 MPs in all!**
- ▶ One double-precision FP unit per SM



GPU vs CPU:

NVIDIA 280 vs Intel i7 860

10

	GPU	CPU ¹
Registers	16,384 (32-bit) / multi-processor ³	128 reservation stations
Peak memory bandwidth	141.7 Gb/sec	21 Gb/sec
Peak GFLOPs	562 (float)/ 77 (double)	50 (double)
Cores	240	4/8 (hyperthreaded)
Processor Clock (MHz)	1296	2800
Memory	1Gb	16Gb
Shared memory	16Kb/TPC ²	N/A
Virtual memory	None	

¹<http://ark.intel.com/Product.aspx?id=41316>

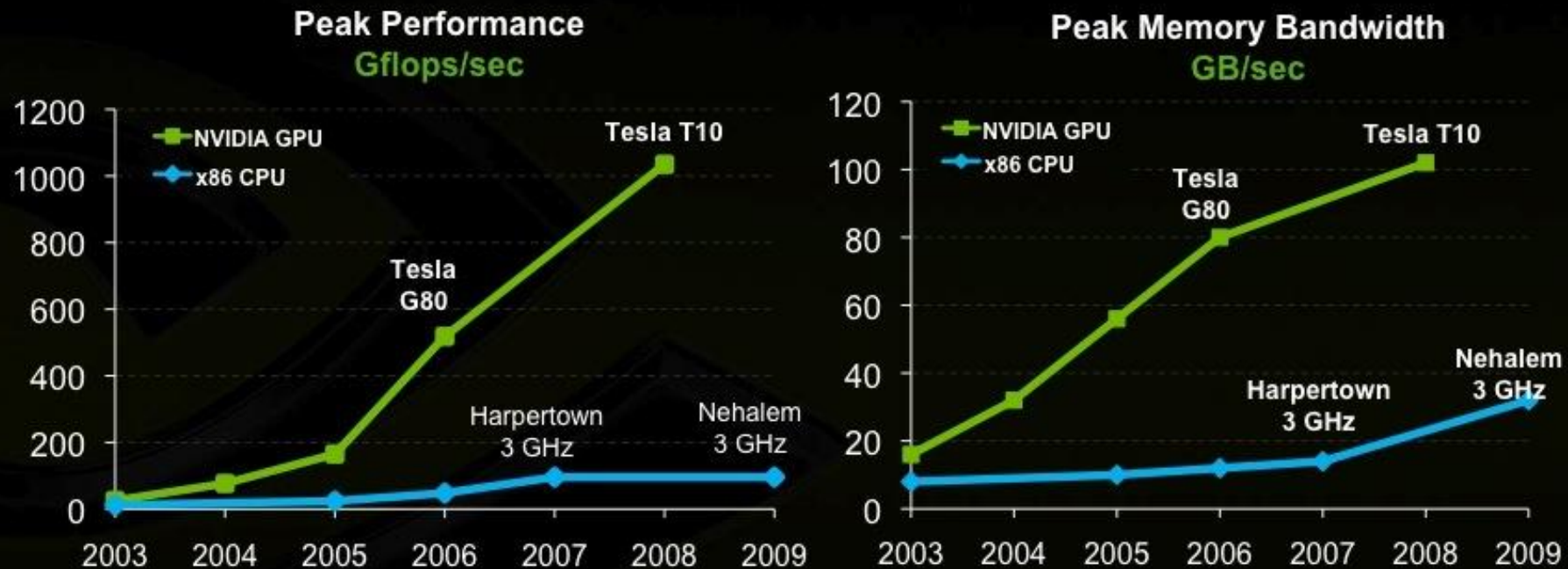
²TPC = Thread Processing Cluster (24 cores)

³30 multi-processors in a 280

GPU vs CPU Peak Performance Trends

11

Increasing Performance Advantage of GPU

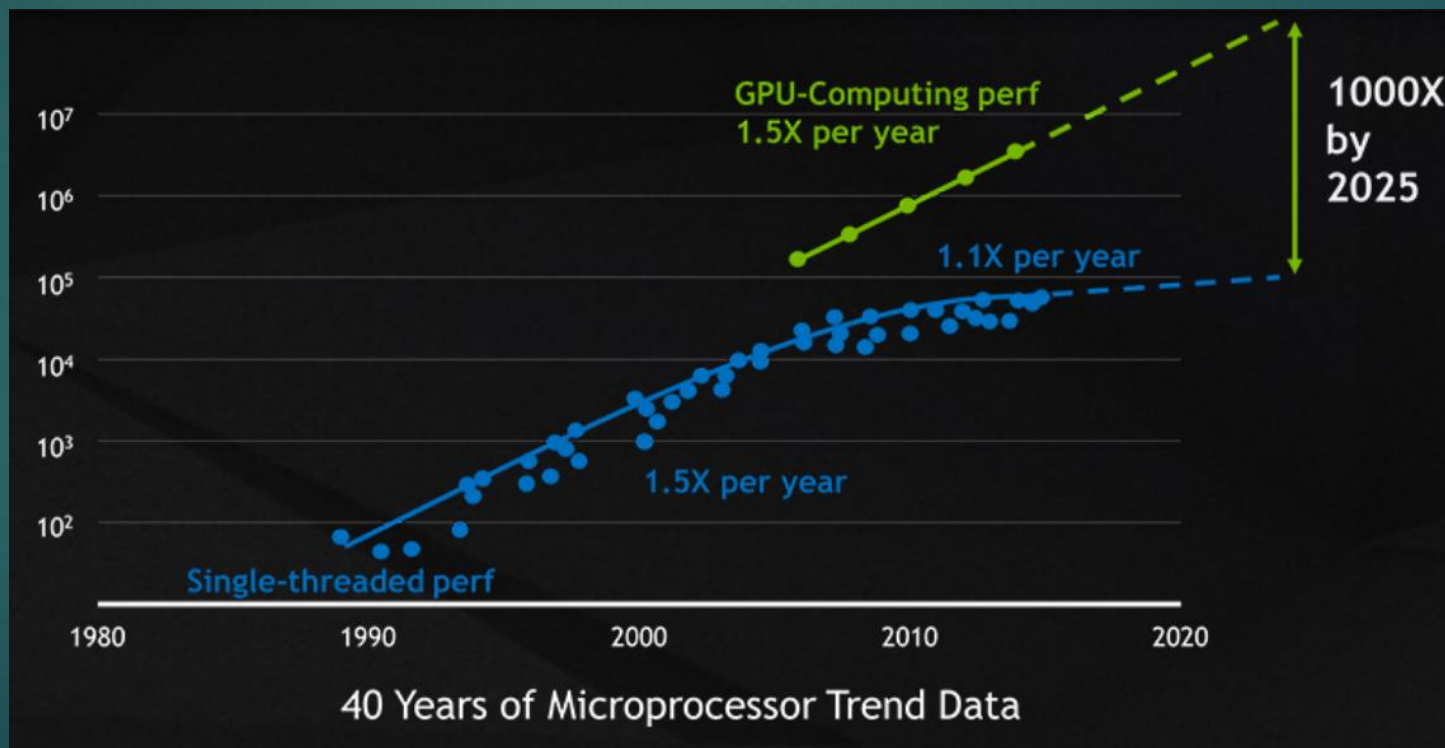


Source : NVIDIA

Peak Performance GPU vs CPU

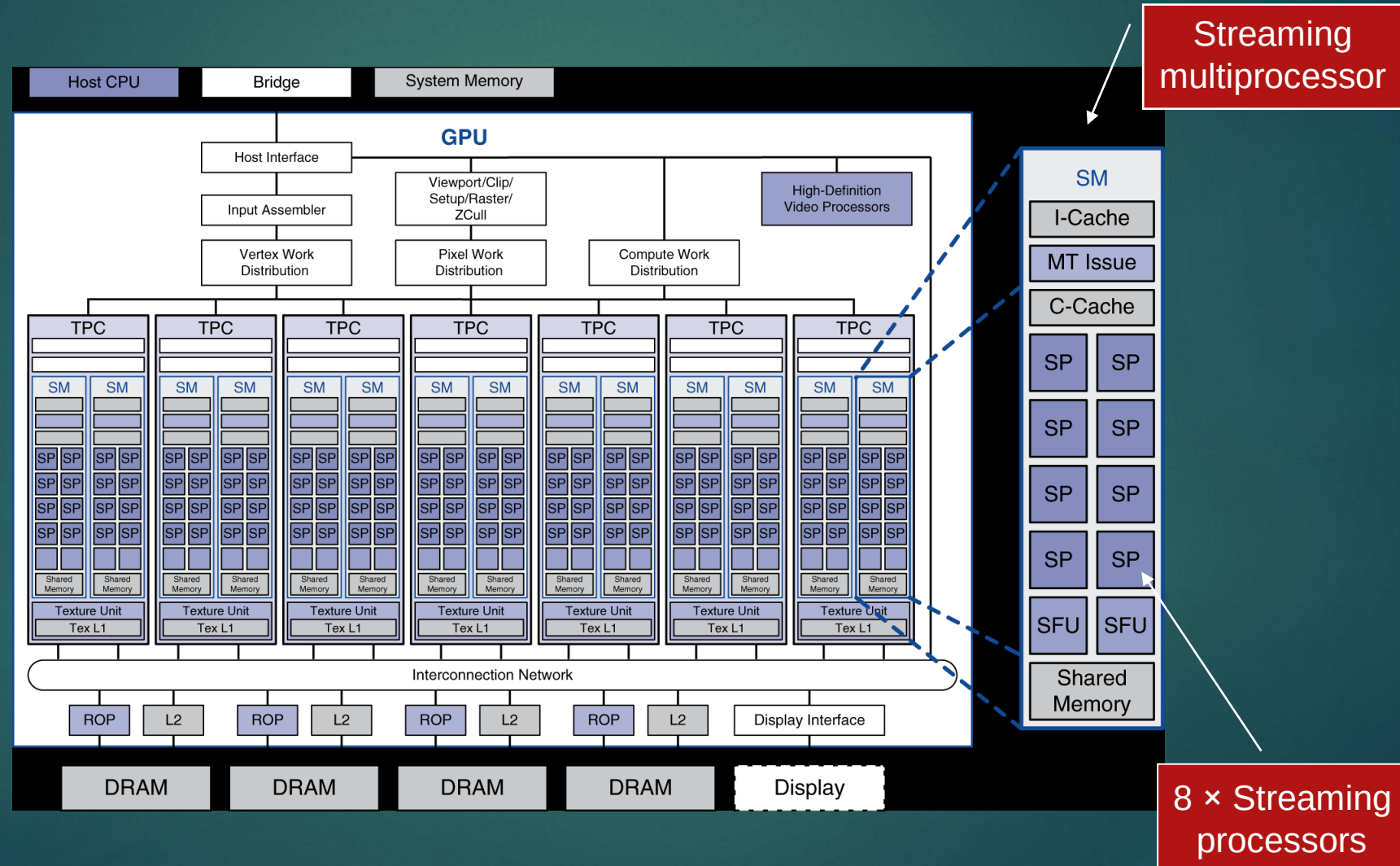
12

	Throughput	Power	Throughput/Power
Intel Skylake	128 SP GFLOPS/4 Cores	100+ Watts	~1 GFLOPS/Watt
NVIDIA V100	15 TFLOPS	200+ Watts	~75 GFLOPS/Watt



GPU Architecture Example: NVIDIA TESLA

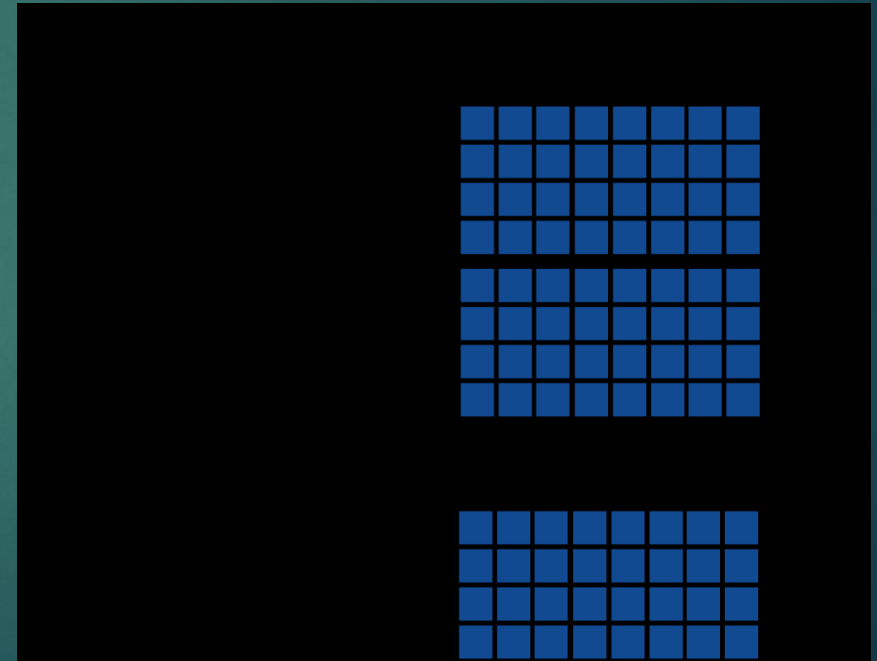
13



GPU Example: NVIDIA Tesla

14

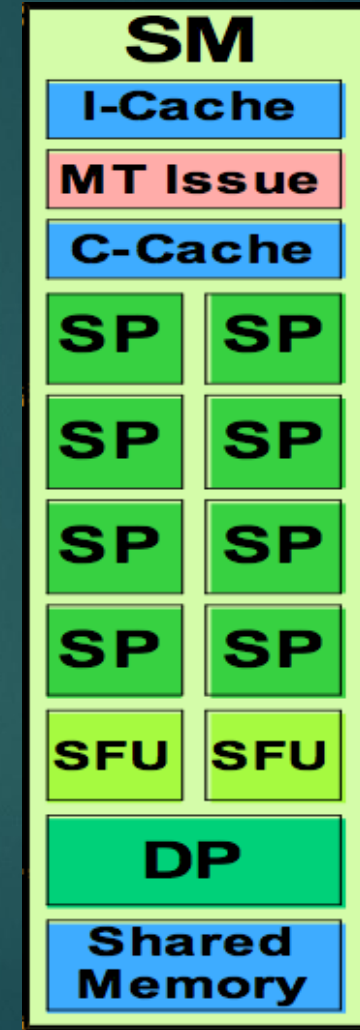
- ▶ Streaming Processors
 - ▶ Single-precision FP and integer units
 - ▶ Each SP is fine-grained multithreaded
- ▶ Warp: group of 32 threads
 - ▶ Executed in parallel, SIMD style
 - ▶ 8 SPs
× 4 clock cycles
 - ▶ Hardware contexts for 24 warps
 - ▶ Registers, PCs, ...



Floating Point

15

- ▶ GPUs are optimized for 32-bit accesses
- ▶ 64-bit double-precision values fetched from memory as **two 32-bit quantities**
 - ▶ May impact performance in the event of memory bank conflicts
- ▶ **One double-precision unit per multi-processor¹**

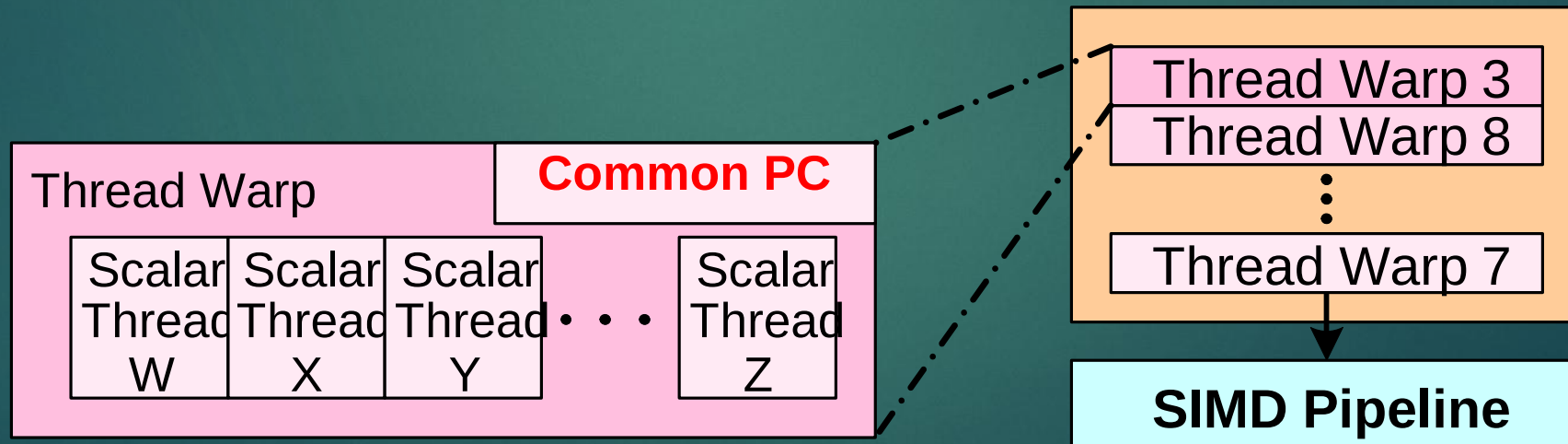


¹<http://www.ddj.com/hpc-high-performance-computing/210102115>

Concept of Programming Model

16

- ▶ SIMT (Nvidia) => Single Instruction Multiple Threads
- ▶ Warp: A set of threads that execute the same instruction (on different data elements)
→ SIMT (Nvidia-speak)
- ▶ All threads run the same kernel
- ▶ Warp: The threads that run lengthwise in a woven fabric ...



Thread Organization

17

- ▶ Threads in a block are partitioned into warps (NVIDIA)
 - ▶ All threads in a warp execute in a Single Instruction Multiple Data, or SIMD, fashion
 - ▶ All paths of conditional branches will be taken
 - ▶ Warp size varies, many graphics cards have 32
- ▶ NO guaranteed execution ordering between warps

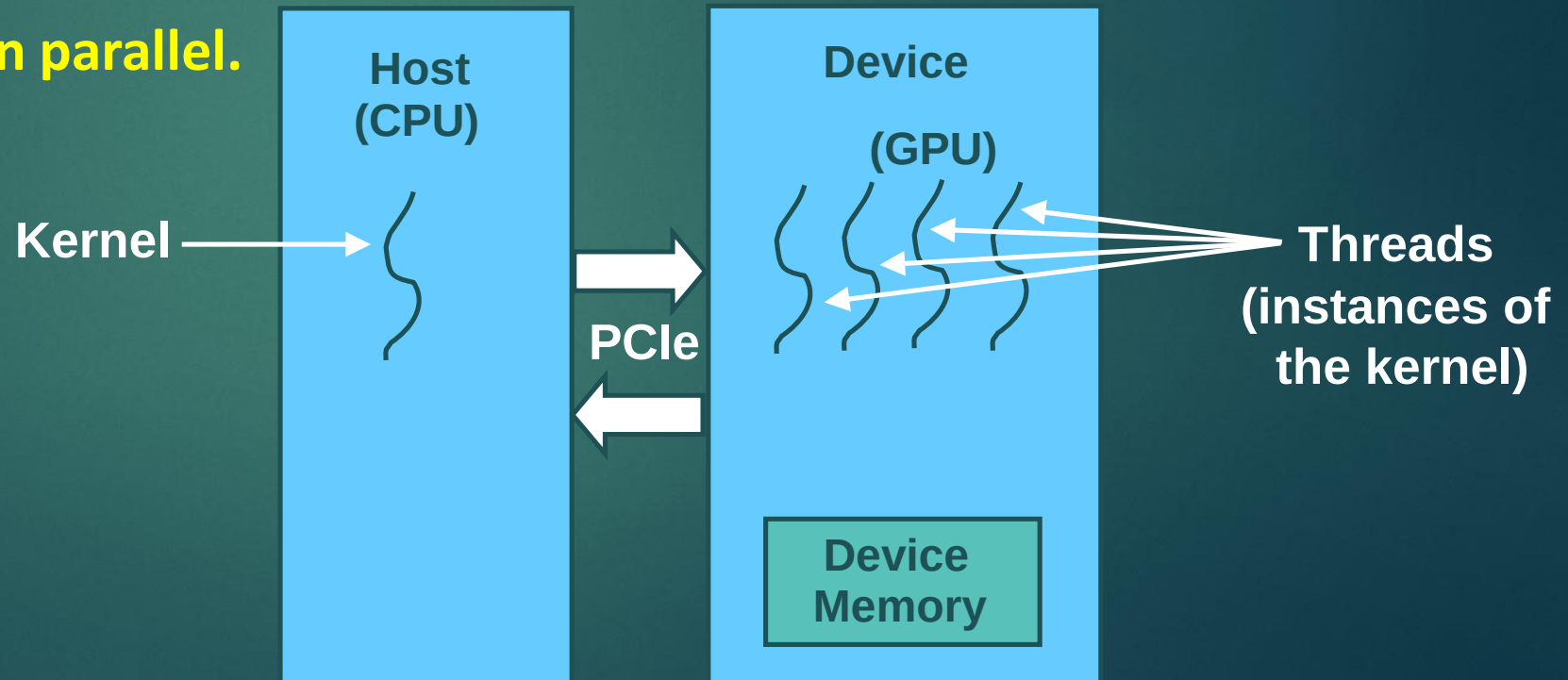
GPU Programming Model

18

- ▶ The GPU is viewed as a compute **device** that:
 - ▶ Is a coprocessor (**slave**) to the CPU (**host**)
 - ▶ Has its own DRAM (**device memory**) but **no virtual memory**

Entire design instance may not fit on the GPU!

- ▶ Kernel is a CPU-callable function. Thread is an instance of a kernel.
- ▶ GPU runs many **threads in parallel**.



GPU Threads vs CPU Threads

19

▶ GPU threads:

- ▶ **Lightweight**, small creation and scheduling overhead, extremely **fast hardware context switching**
- ▶ **Need to issue 1000s of GPU threads** to hide global memory latencies (600-800 cycles)

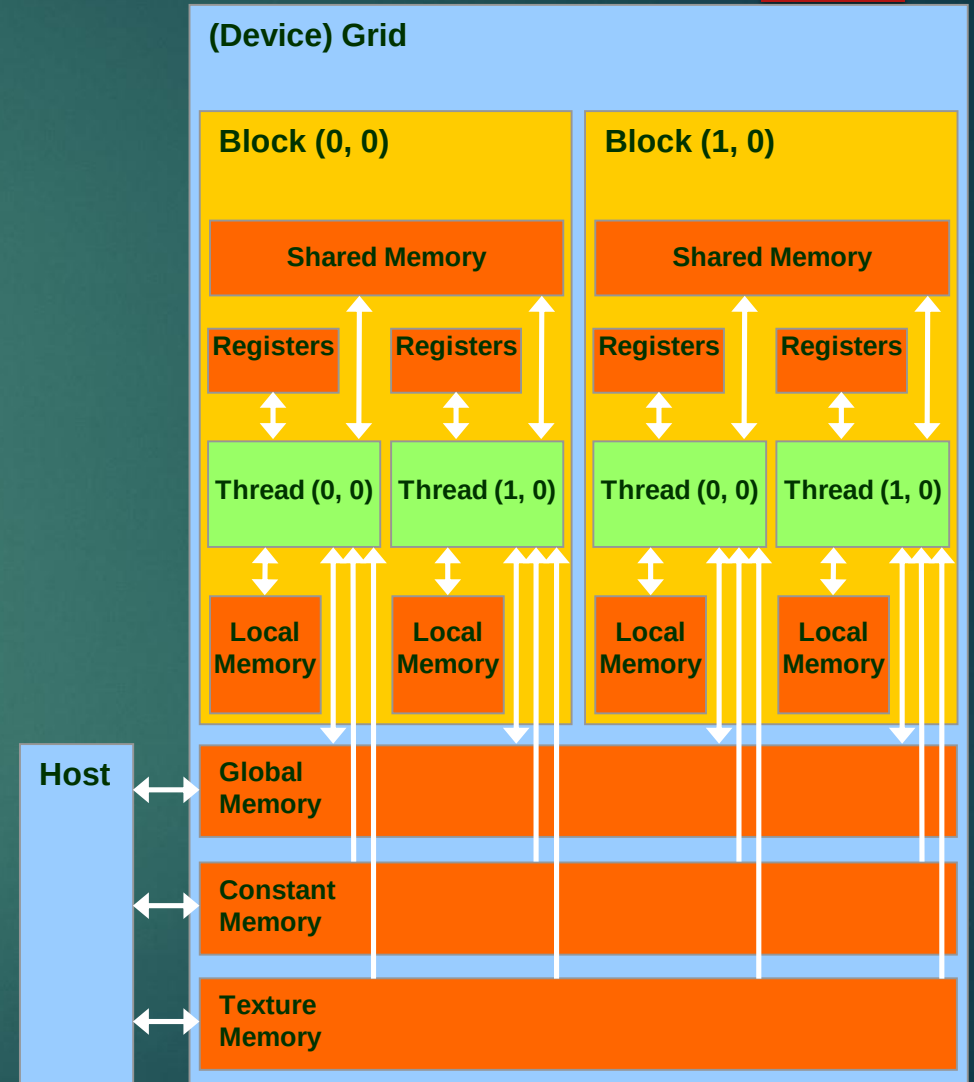
▶ CPU threads:

- ▶ Heavyweight, large scheduling overhead, **slow context switching**
- ▶ **Multi-GPU usage** requires invocation of multiple CPU threads
 - ▶ Each CPU thread creates a GPU context
 - ▶ Context swapping is required for a CPU thread to access GPU memory allocated by another CPU thread

Device Memory Space

20

- ▶ Each thread runs on a SP and has:
 - ▶ R/W per-thread **registers (on-chip)**
 - ▶ Limit usage (max 16K/MP)
 - ▶ R/W per-thread **local memory (off)**
 - ▶ R/W per-block **shared memory (on)**
 - ▶ Need to avoid bank conflicts
 - ▶ R/W per-grid **global memory (off)**
 - ▶ Not cached, 600-800 cycle read
 - ▶ Latency hidden by parallelism
 - ▶ and fast context switches
 - ▶ Main means for data transfer from host and device
 - ▶ Coalescing recommended
 - ▶ RO per-grid cached **constant and texture memory (off)**
- ▶ The host can R/W **global, constant** and **texture** memories (visible to all threads)



Data Transfers (CPU↔GPU)

21

- ▶ GPUs and CPUs communicate via a PCIe bus

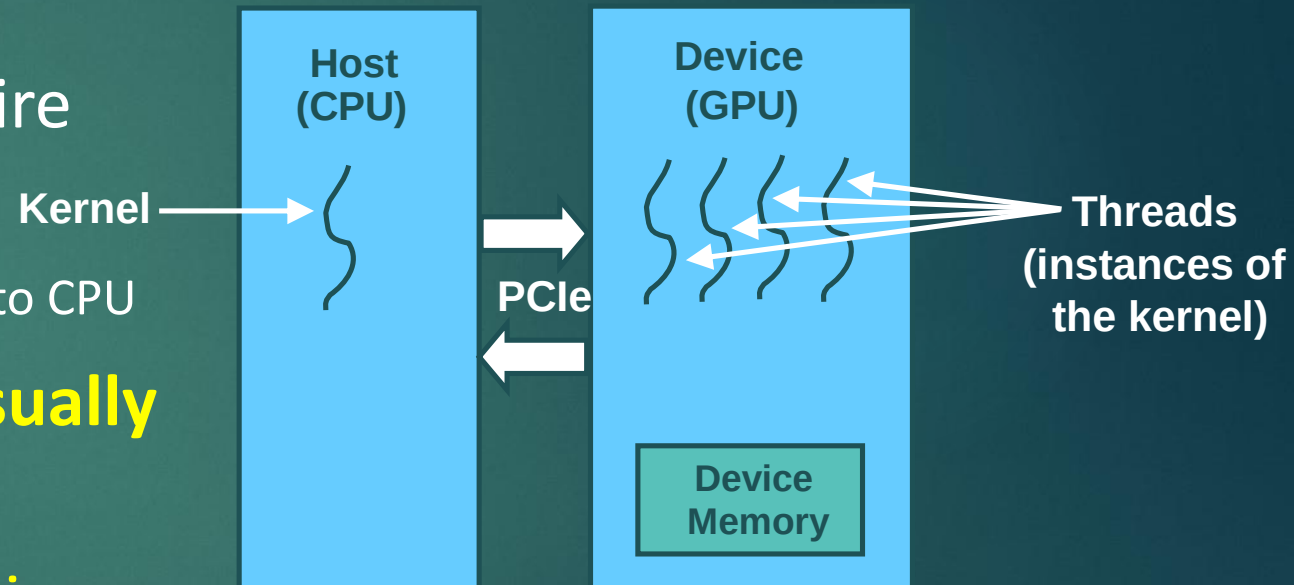
- ▶ This **communication is expensive** and should be minimized for target applications

- ▶ Graphics applications usually require

- ▶ Initial data to be sent from CPU to GPU
 - ▶ Single transfer of processed data from GPU to CPU

- ▶ **General purpose computations usually require**

- ▶ **Multiple transfers between CPU and GPU (since conditional checks on CPU)**
 - ▶ Possibility of saturating the PCIe bus and reducing the achievable performance

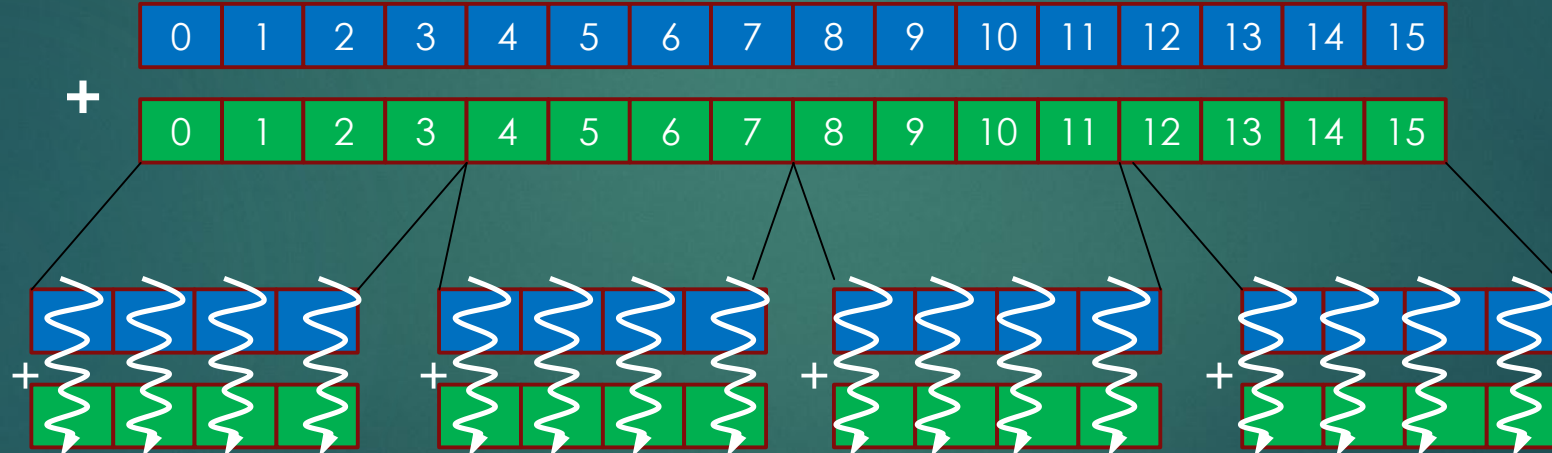


Multiple-Thread Memory Access

22

- ▶ Same instruction in different threads uses thread id to index and access different data elements

Let's assume $N=16$, $\text{blockDim}=4 \rightarrow 4$ blocks



Sample GPU Multiple-Thread Code (Simplified)

23

CPU code

```
for (ii = 0; ii < 100; ++ii) {  
    C[ii] = A[ii] + B[ii];  
}
```



CUDA code

```
// there are 100 threads  
__global__ void KernelFunction(...) {  
    int tid = blockDim.x * blockIdx.x + threadIdx.x;  
    int varA = aa[tid];  
    int varB = bb[tid];  
    C[tid] = varA + varB;  
}
```

Sample GPU Program (Less Simplified)

24

CPU Program

```
void add_matrix
( float *a, float* b, float *c, int N) {
    int index;
    for (int i = 0; i < N; ++i)
        for (int j = 0; j < N; ++j) {
            index = i + j*N;
            c[index] = a[index] + b[index];
        }
}

int main () {

    add_matrix (a, b, c, N);
}
```

GPU Program

```
__global__ add_matrix
( float *a, float *b, float *c, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    int index = i + j*N;
    if (i < N && j < N)
        c[index] = a[index]+b[index];
}

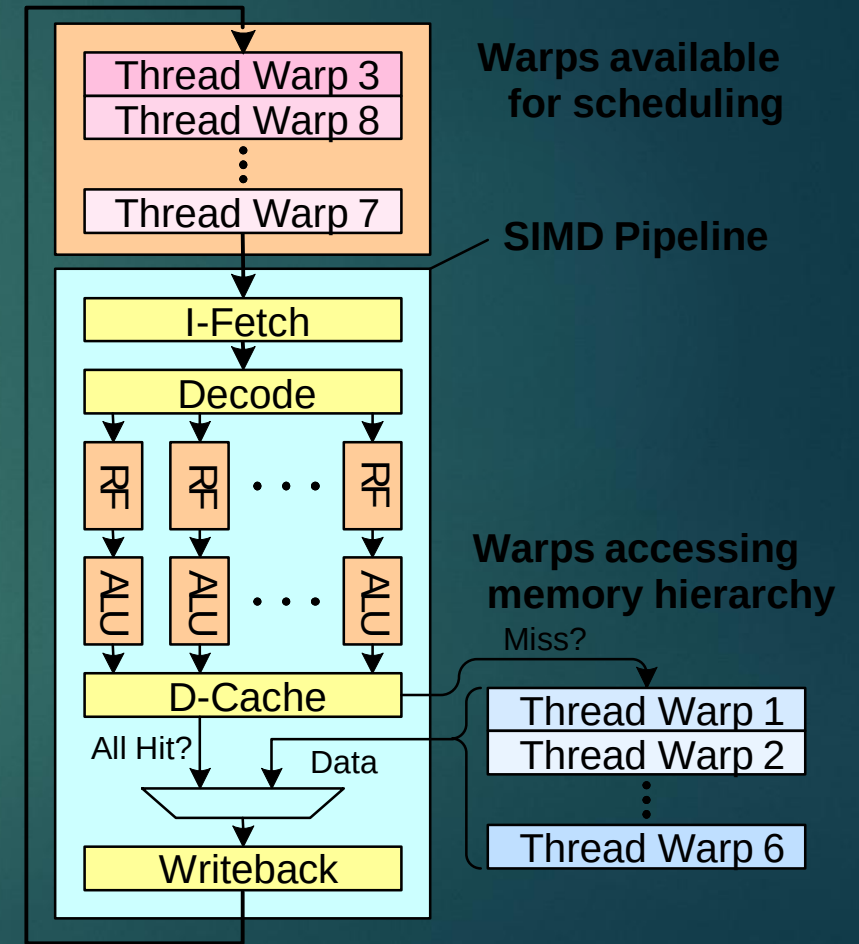
int main() {
    dim3 dimBlock( blocksize, blocksize) ;
    dim3 dimGrid (N/dimBlock.x, N/dimBlock.y);
    add_matrix<<<dimGrid, dimBlock>>>( a, b, c, N);
}
```

24

Thread Execution

25

- ▶ Warp: A set of threads that execute the same instruction (on different data elements)
- ▶ Fine-grained multithreading
 - ▶ One instruction per thread in pipeline at a time (No branch prediction)
 - ▶ Interleave warp execution to hide latencies
- ▶ Register values of all threads stay in register file
- ▶ No OS context switching
- ▶ Memory latency hiding
 - ▶ Graphics has millions of pixels



Thread Execution

26

▶ CPU

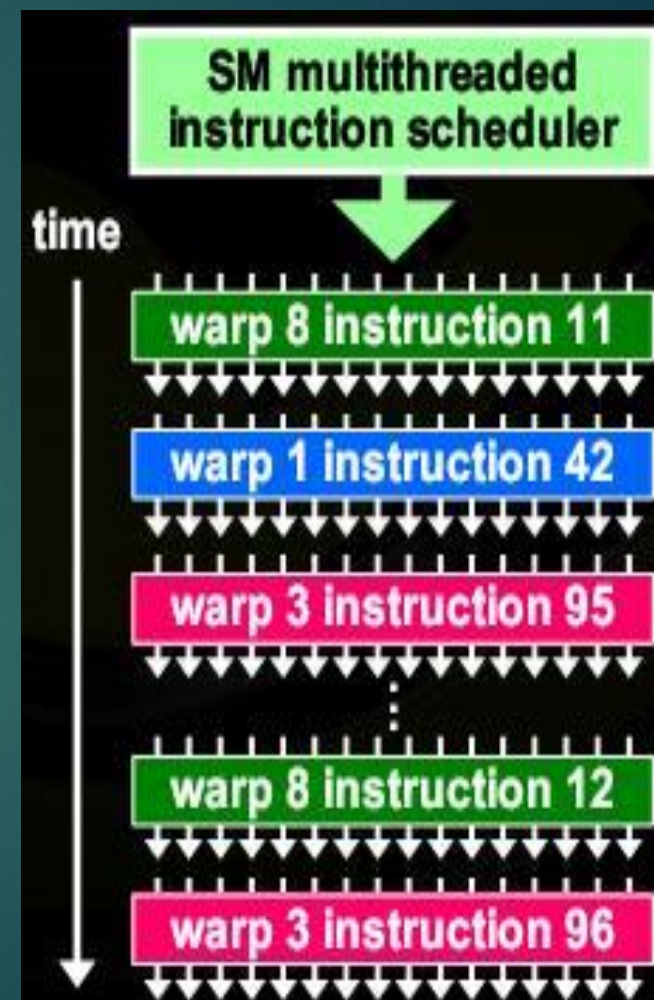
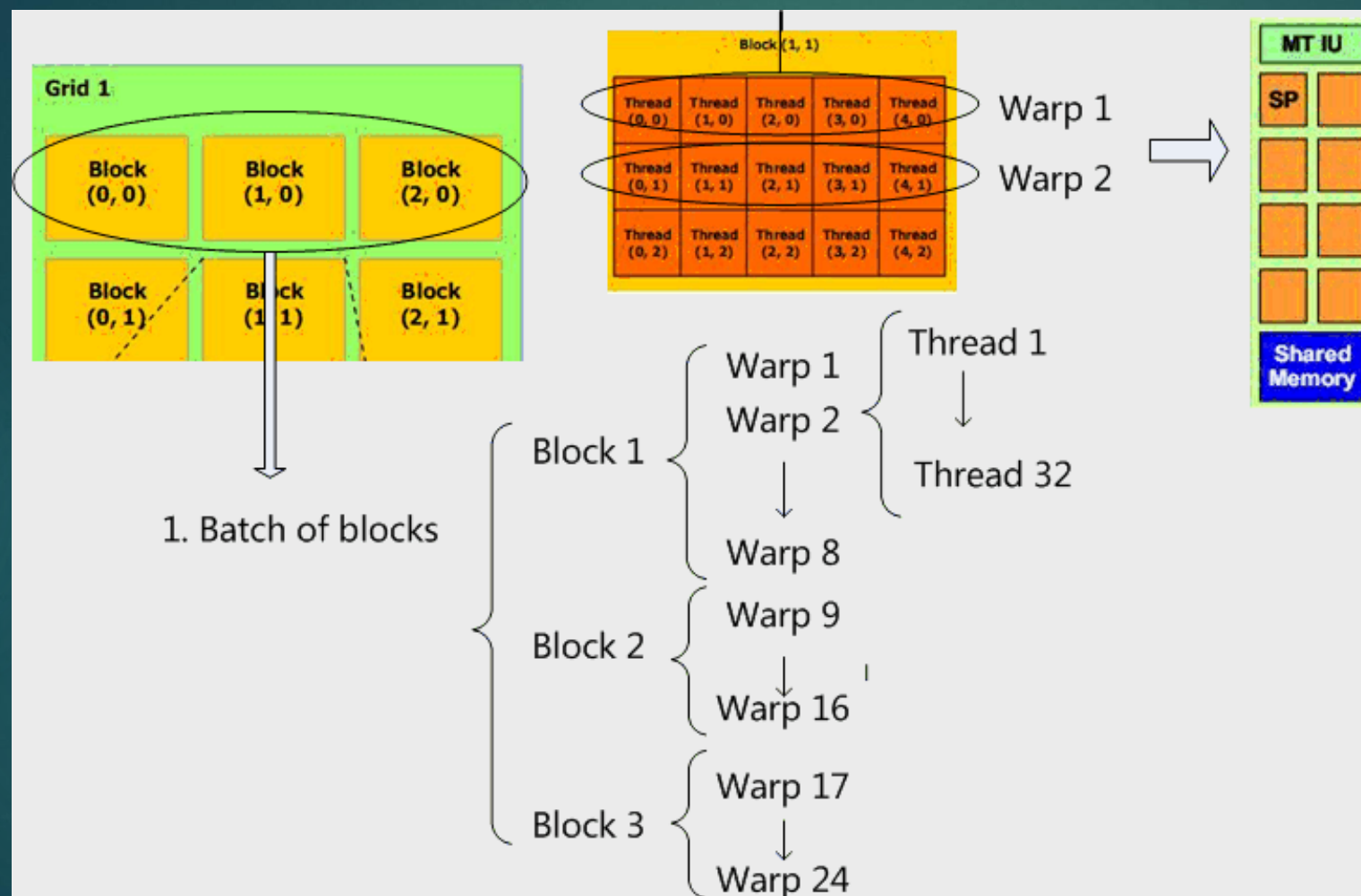
- ▶ All **threads are equivalent**
- ▶ Read/write concurrently to the same memory
- ▶ Synchronization primitives required to avoid collisions

▶ GPU (NVIDIA)

- ▶ Each **CPU thread maintains a unique context**
- ▶ GPU resources (e.g. memory, code modules, address space) are context-specific
- ▶ Each CPU thread can access a single context at once
- ▶ **Contexts must be exchanged between CPU threads to share GPU resources** between CPU threads
- ▶ Contexts use reference counting and are automatically destroyed

Programming Execution Model

27



SIMD Conditional Processing

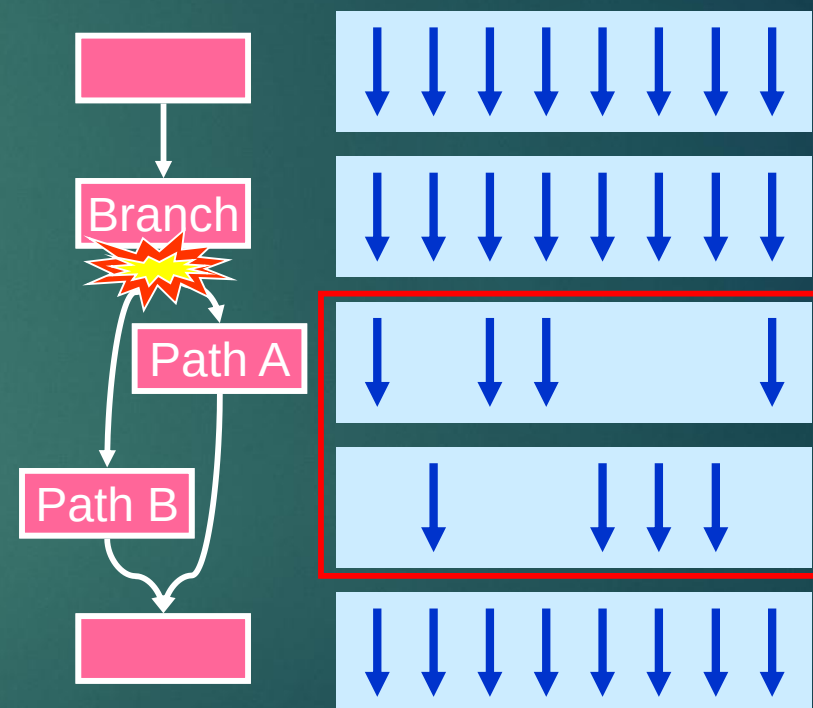
28

- ▶ Unlike threads in a CPU-based program, **SIMD programs cannot follow different execution paths**
- ▶ **Ideal scenario:**
 - ▶ **All GPU threads follow the same execution path**
 - ▶ **All processors active continuously**
- ▶ In divergent paths, some processors execute the then-block and others the else-block
 - ▶ Program flow cannot actually diverge. All instructions are executed
 - ▶ **The then- and else- blocks are both executed**
 - ▶ A bit is used to enable/disable processors based on the block being executed
- ▶ **Parallelism is reduced**, impacting performance

Control Flow Problem in GPUs/SIMD

29

- ▶ GPU uses SIMD pipeline to save area on control logic.
 - ▶ Group scalar threads into warps
- ▶ Branch divergence occurs when threads inside warps branch to different execution paths.



Branch Divergence

30

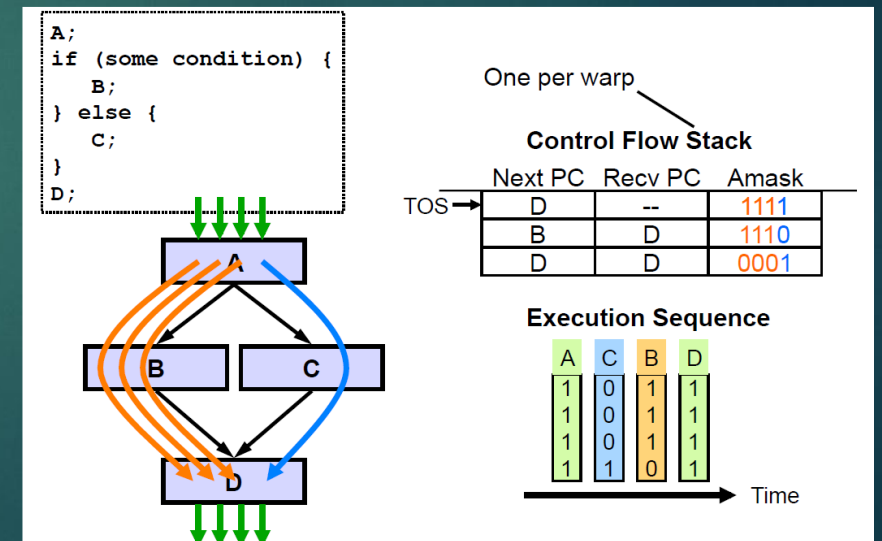
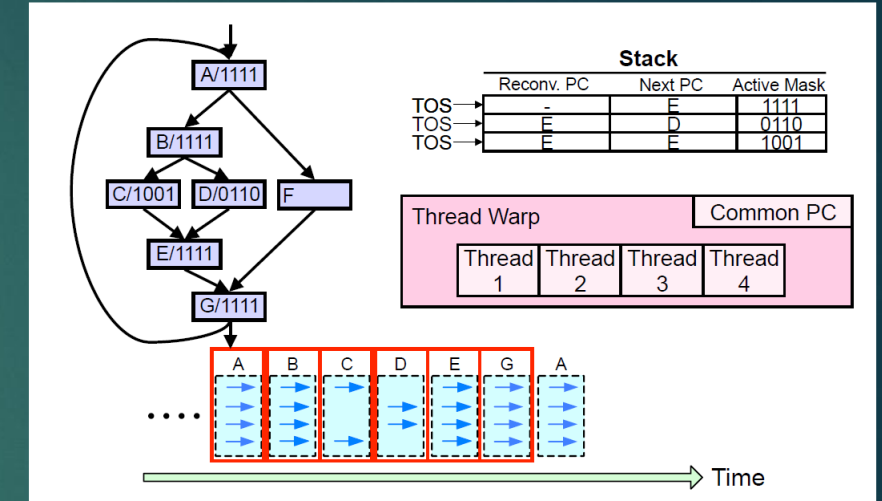
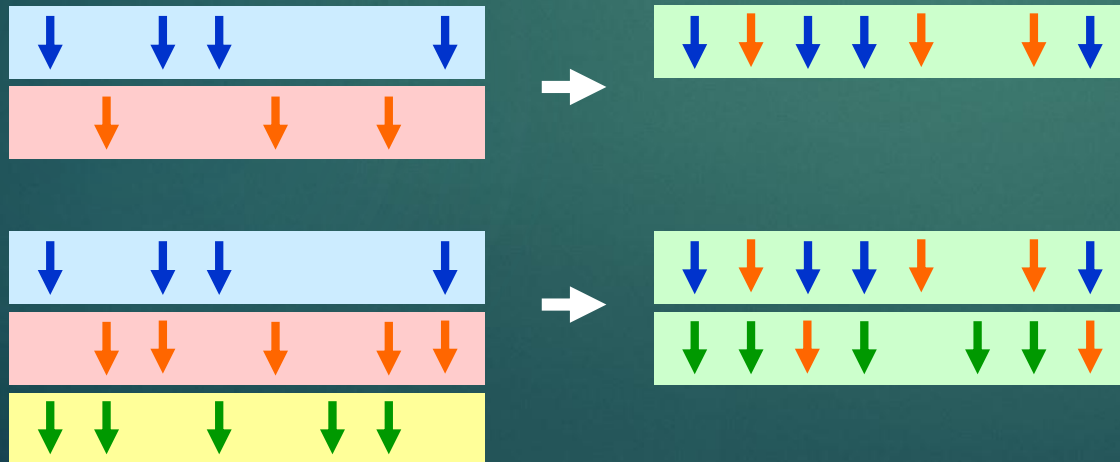
- ▶ Threads in one warp execute very different branches
- ▶ Significantly harms the performance!
- ▶ Simple solution:
 - ▶ Reordering the threads so that all threads in each block are more likely to take the same branch
 - ▶ Not always possible

Branch Divergence Handlings

31

- ▶ Reduce Branch Divergence
- ▶ Execution Mask
- ▶ Dynamic Warp Formation
 - ▶ Idea: Dynamically merge threads executing the same instruction (after branch divergence)
 - ▶ Form new warp at divergence
 - ▶ Enough threads branching to each path to create full new warps

Others



Idle Processors

- ▶ Idle CPU processors can be **dynamically rescheduled** by OS
- ▶ SIMD processors are not actually idle
 - ▶ All processors scheduled are following identical execution paths
 - ▶ **Disabled (idle) processors are unavailable** for other work and cannot be rescheduled
 - ▶ Effective utilization of processors is the programmer's responsibility
 - ▶ **Scheduling is an art, not necessarily a science**
 - ▶ Techniques will vary from chip to chip

Cost of Memory Access

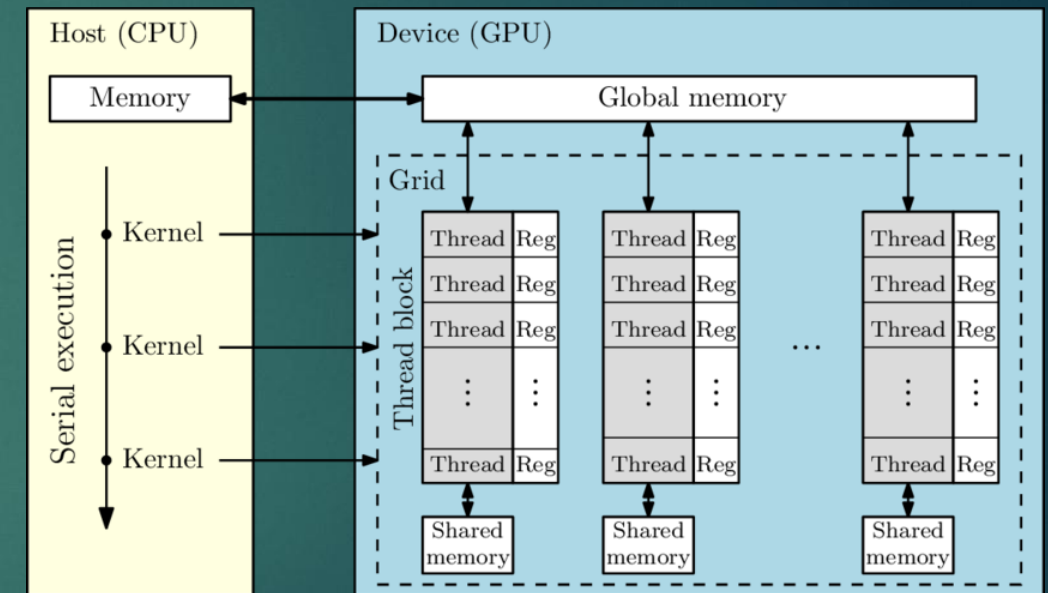
33

- ▶ Registers are extremely fast, but are a limited resource
- ▶ Cached memories also tend to be small
- ▶ For large data sets, **global memory provides read & write access**
 - ▶ Accesses take between 600 and 800 clock cycles
 - ▶ **Accesses are *not* cached**
 - ▶ To hide memory latency, the hardware provides **fast context switches when memory is accessed**
 - ▶ However, there must be enough computational work to do to hide the high cost of memory access
- ▶ Programmers need to be smart
 - ▶ Compilers often don't provide the necessary optimizations when optimizing for speed instead of code size
 - ▶ It can sometimes be **cheaper to recompute a result** than perform a memory read/write

GPU Code Development and Execution

34

- ▶ Source Codes => Host CPU side and GPU side
- ▶ Host Compiler => CPU side
- ▶ Device Compiler => GPU side
- ▶ Linker => Executable Codes
- ▶ Code Loader => Load into
 - ▶ Host Memory and Device Memory
- ▶ Code Execution => Activate Program
 - ▶ => Threads



GPU Code Development and Execution

35

► CUDA Code Example

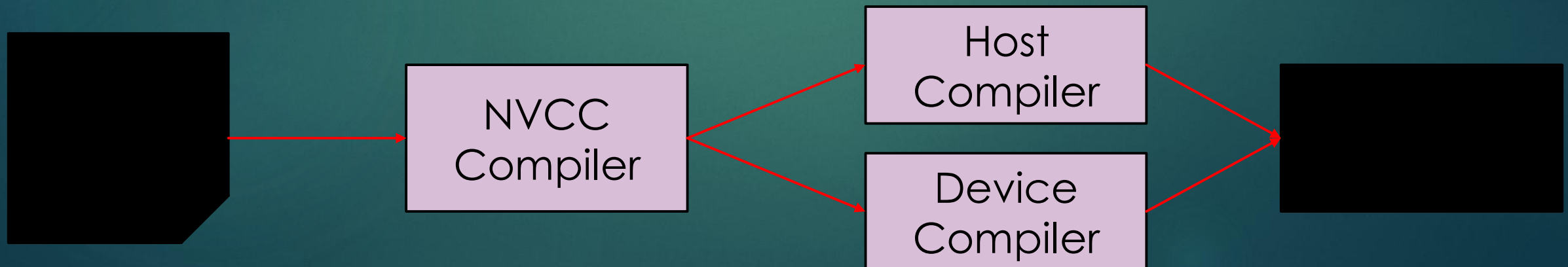
CPU side

```
int main()
{
    ...
    // Kernel invocation with N threads
    VecAdd<<<1, N>>>(A, B, C);
    ...
}
```

Asynchronous call

GPU side

```
// Kernel definition
__global__ void VecAdd(float* A, float* B, float* C)
{
    int i = threadIdx.x;
    C[i] = A[i] + B[i];
}
```



CUDA API Development

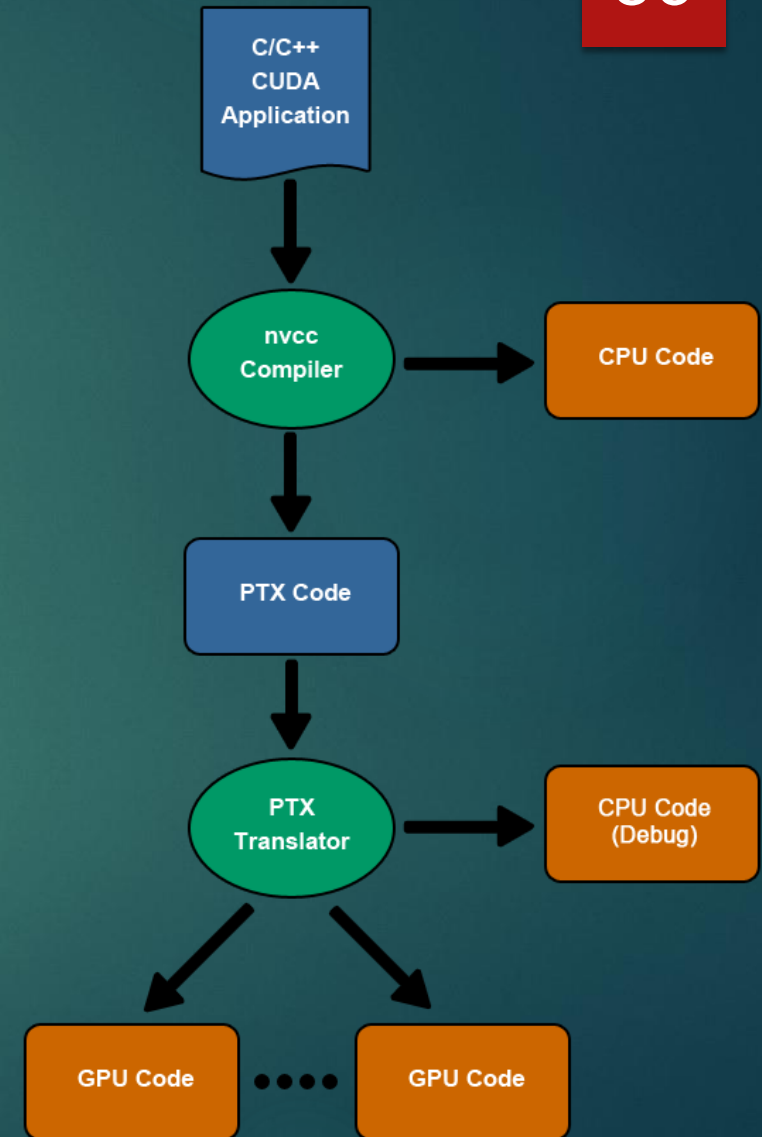
36

- ▶ The CUDA source file uses `.cu` as extension. It contains host and device source codes.

The CUDA Compiler Driver `nvcc` can compile it and generate CPU/PTX binary code.

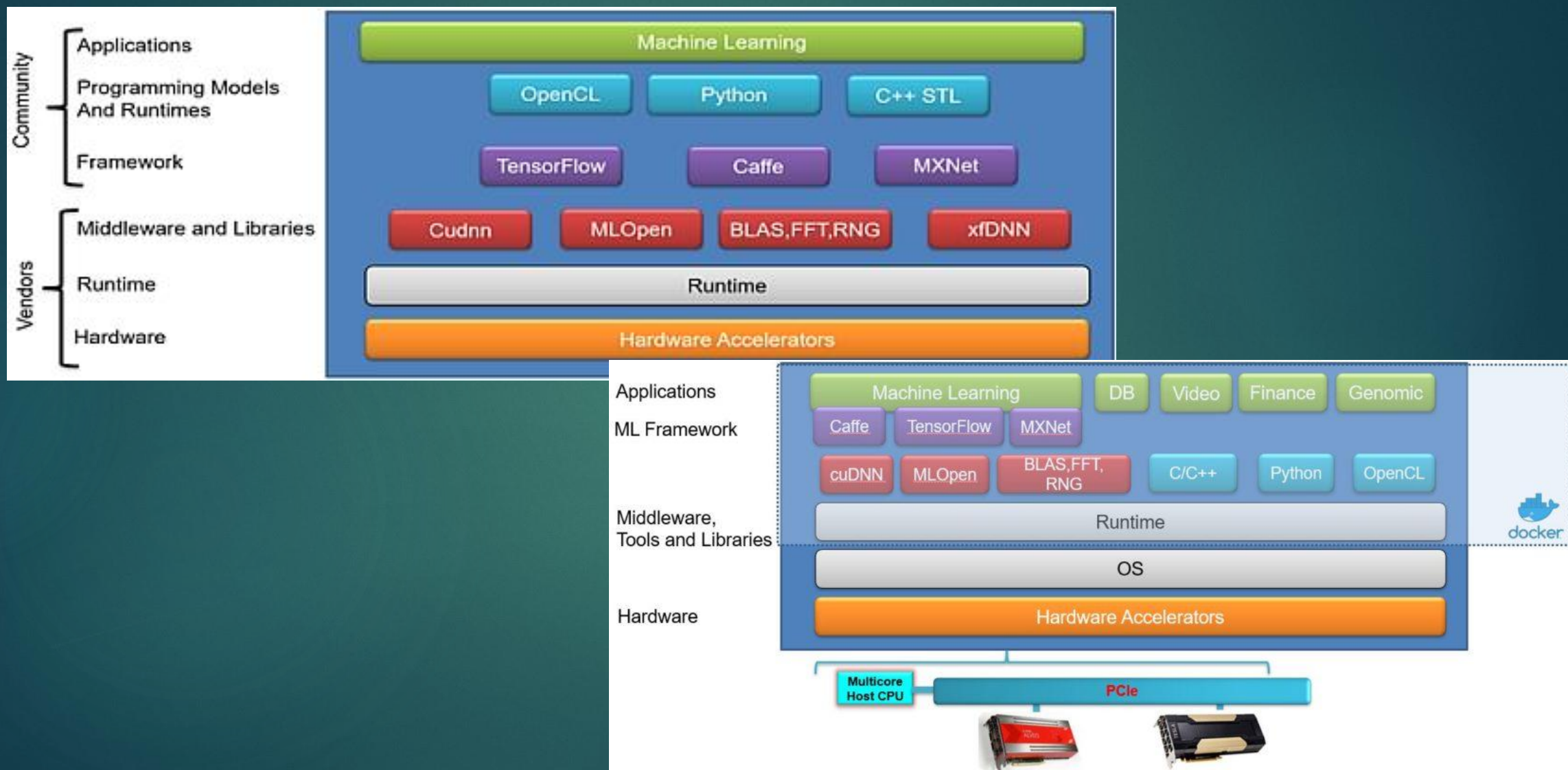
(PTX: Parallel Thread Execution, a device independent VM code)

PTX code may be further translated for special GPU-Arch.



GPU Software Stack

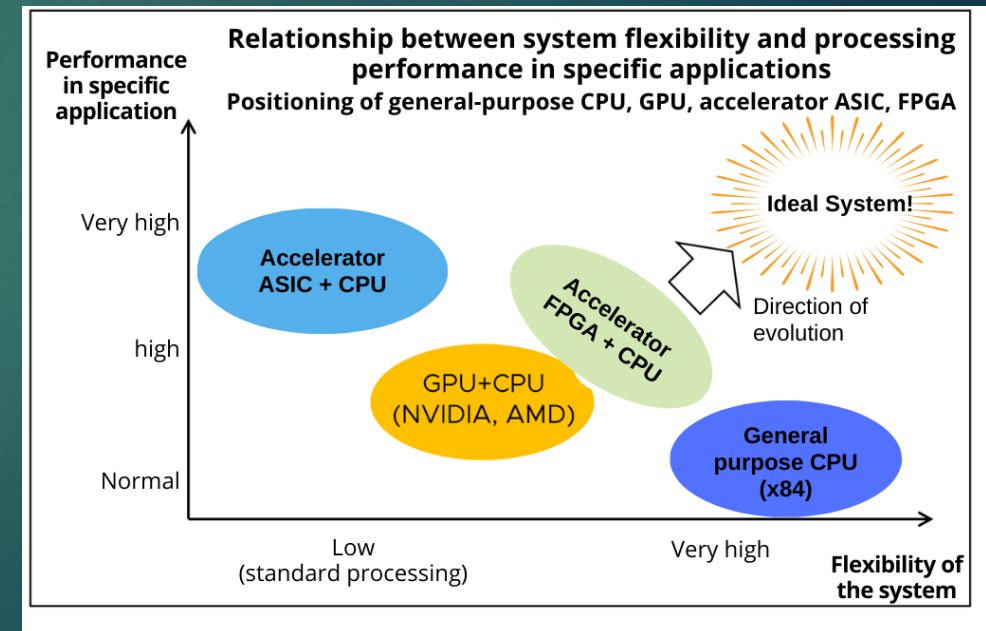
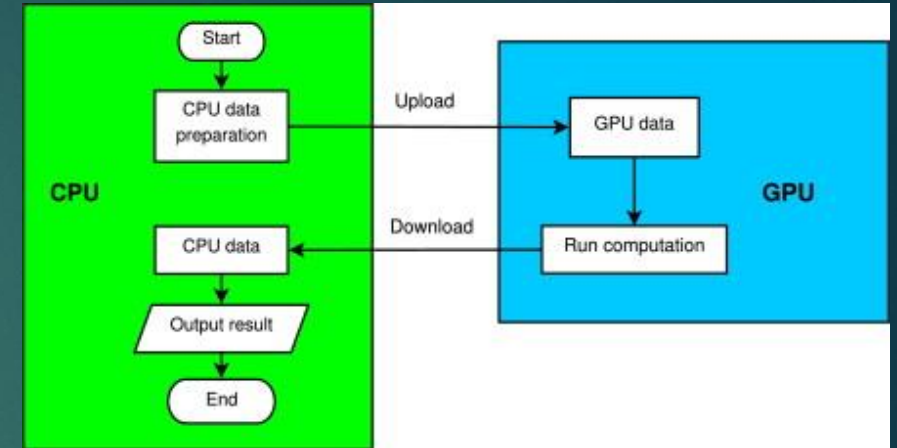
37



Conclusion

38

- ▶ Start from Video/Graphics Processing
- ▶ SIMD => Massively Parallel Processors
- ▶ General Purposed GPU
- ▶ High Performance, But High Power
- ▶ Computing Accelerators
 - ▶ Host Control
 - ▶ Data Transfer
 - ▶ Processing in GPU Memory



END

QUESTIONS & ANSWERS